



API REST SID CFE-LAPEM

Curso Teórico-Práctico

Manual para Desarrolladores

Junio 2026

Laboratorio de Pruebas de Equipos y Materiales — LAPEM

i Este manual está diseñado para desarrolladores. Las prácticas se implementan en Visual Studio Code usando C# (.NET 8).



1. Repositorio GitHub — Configuración Inicial

¡ Configure su repositorio antes de iniciar cualquier práctica. Todo el código del curso debe vivir en ese repositorio.

1. Iniciar sesión en <https://github.com>
2. Crear un nuevo repositorio: nombre SID-API-Client, visibilidad Private.
3. Marcar 'Add a README file' → 'Create repository'.
4. Ir a Settings → Collaborators → 'Add people'.
5. Agregar colaborador: casolism (rol: Read).
6. Agregar colaborador: dsanchezm19 (rol: Read).
7. Clonar el repositorio en su equipo:

```
git clone https://github.com/<su-usuario>/SID-API-Client.git  
cd SID-API-Client
```

⚠ Nota: El repositorio debe ser privado y compartido con casolism y dsanchezm19 ANTES de iniciar la Práctica 1. Se verificará durante la revisión del curso.

Convención de commits:

- Un commit por práctica completada.
- Formato del mensaje: P{N}: <descripción breve> Ejemplo: P1: AuthService y configuración inicial

1. Base Teórica

1.1 ¿Qué es el SID?

El Sistema de Inspección a Distancia (SID) es una plataforma desarrollada por CFE-LAPEM que permite la supervisión y validación remota de los procesos de fabricación y pruebas de materiales y equipos eléctricos.

Principales beneficios:

- Reducción de tiempos de inspección física.
- Menores costos logísticos para fabricantes e inspectores.
- Mayor trazabilidad documental del proceso.
- Acceso remoto en tiempo real a resultados de pruebas.
- Gestión centralizada de expedientes y avisos de prueba.

1.2 Proceso de Inspección — Fases F0 a F4

F0 — Acceso: Autenticación JWT y configuración inicial de parámetros de la empresa.

F1 — Configuración Inicial: Registro de instrumentos, prototipos, valores de referencia, productos, normas y tipos de prueba.

F2 — Preparación y Fabricación: Contratos CFE, órdenes de fabricación y expedientes con sus muestras.

F3 — Pruebas: Registro de resultados, verificación de pruebas no satisfactorias y validación del expediente.

F4 — Liberación: Generación y consulta de avisos de prueba para firma electrónica.

1.3 Arquitectura del API REST

i URL base de capacitación: https://lapem.cfe.gob.mx/sid_capacitacion

i Swagger UI: https://lapem.cfe.gob.mx/sid_capacitacion/swagger

Autenticación — todas las llamadas (excepto /Login) requieren:

```
Authorization: Bearer <token_jwt>
```

El token se obtiene en POST /F0_Acceso/Login y su vigencia puede verificarse con POST /F0_Acceso/ValidarToken.

1.4 Flujo Completo de Inspección

8. Autenticarse (F0 Login).
9. Configurar parámetros de la empresa (muestreo, intentos permitidos).
10. Registrar catálogos F1: instrumentos, prototipos, valores de referencia, productos, normas y pruebas.



11. Registrar contrato CFE (F2).
12. Crear orden de fabricación asociada al contrato (F2).
13. Generar expediente de pruebas con sus muestras (F2).
14. Registrar resultados de prueba para cada muestra (F3).
15. Validar que el expediente esté completo (F3).
16. Terminar pruebas del expediente (F3).
17. Generar aviso de prueba (F4).
18. Consultar y firmar el aviso de prueba (F4).

1.5 Manejo de Errores HTTP

- 200 OK — Operación exitosa. Procesar la respuesta normalmente.
- 400 Bad Request — Datos inválidos. Revisar el cuerpo de la solicitud.
- 401 Unauthorized — Token vencido. Renovar con /F0_Acceso/Login.
- 409 Conflict — Registro duplicado. Verificar si el recurso ya existe.
- 500 Internal Server Error — Error del servidor. Reportar a soporte.

1.6 Cambio Importante en el API — Mayo 2026

⚠ Nota: A partir de la versión del 21-May-2026, todos los endpoints POST que crean registros retornan únicamente un string con el ID del recurso generado. Ya no retornan un objeto completo ni un booleano.

Ejemplo:

```
// Respuesta anterior: { "id": "abc123", "nombre": "...", ... }  
// Respuesta nueva:   "abc123" ← solo el ID como string
```



3. Prácticas de Desarrollo en Visual Studio Code

⚠ Nota: A partir del 21-May-2026 todos los POST que crean registros retornan el ID generado como string. Use Task<string?> en los métodos de creación, no Task<bool>.

Práctica 1: Autenticación y Configuración Inicial (paso a paso detallado)

Objetivo:

Crear la estructura completa del proyecto y el servicio de autenticación JWT que servirá como base para todas las prácticas siguientes.

Endpoints:

POST /F0_Acceso/Login
GET /F0_Acceso/Configuracion
POST /F0_Acceso/Configuracion
POST /F0_Acceso/ValidarToken
POST /F1_ConfiguracionInicial/EstadoSID

Paso 1 — Crear el proyecto

En la carpeta del repositorio clonado, abrir la terminal integrada de VS Code (Ctrl+) y ejecutar:

```
dotnet new sln -n SidApiClient
dotnet new console -n SidApiClient.App
dotnet new classlib -n SidApiClient.Core
dotnet sln add SidApiClient.App/SidApiClient.App.csproj
dotnet sln add SidApiClient.Core/SidApiClient.Core.csproj
cd SidApiClient.App
dotnet add reference ../SidApiClient.Core/SidApiClient.Core.csproj
```

Instalar paquetes NuGet:

```
dotnet add ../SidApiClient.Core package Newtonsoft.Json
dotnet add ../SidApiClient.Core package Microsoft.Extensions.Http
```

Paso 2 — Estructura de carpetas

En SidApiClient.Core crear las carpetas: Models/DTOs, Services, Helpers.

```
SidApiClient.Core/
├── Models/DTOs/
│   ├── LoginRequest.cs
│   ├── ConfiguracionEmpresaDTO.cs
│   └── MuestreoDTO.cs
├── Services/
│   └── AuthService.cs
└── Helpers/
    └── ApiClient.cs
```

Paso 3 — appsettings.json

Crear SidApiClient.App/appsettings.json:

```
{"SidApi":{"BaseUrl":"https://lapem.cfe.gob.mx/sid_capacitacion","Username":"tu_usuario","Password":"tu_contraseña"}}
```

Paso 4 — DTOs de autenticación

LoginRequest.cs:

```
namespace SidApiClient.Core.Models.DTOs;

public class LoginRequest
{
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}
```

MuestreoDTO.cs:

```
namespace SidApiClient.Core.Models.DTOs;

public class MuestreoDTO
{
    public string TipoMuestreo { get; set; } = string.Empty;
    public int NumeroFijoMuestras { get; set; }
}
```

ConfiguracionEmpresaDTO.cs:

```
namespace SidApiClient.Core.Models.DTOs;

public class ConfiguracionEmpresaDTO
{
    public string? Id { get; set; }
    public int NumeroIntentosPermitidos { get; set; }
    public MuestreoDTO? Muestreo { get; set; }
}
```

Paso 5 — ApiClient.cs (cliente HTTP base con JWT)

```
using System.Net.Http.Headers;
using System.Text;
using Newtonsoft.Json;

namespace SidApiClient.Core.Helpers;

public class ApiClient
{
    private readonly HttpClient _http;

    public ApiClient(string baseUrl) =>
        _http = new HttpClient { BaseAddress = new Uri(baseUrl) };

    public void SetToken(string token) =>
        _http.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", token);

    public async Task<T?> GetAsync<T>(string url)
    {
        var r = await _http.GetAsync(url);
        r.EnsureSuccessStatusCode();
        return JsonConvert.DeserializeObject<T>(await r.Content.ReadAsStringAsync());
    }
}
```

```
// POST que retorna el ID creado como string (cambio API May-2026)
public async Task<string?> PostAsync<T>(string url, T body)
{
    var r = await _http.PostAsync(url, Json(body));
    r.EnsureSuccessStatusCode();
    return await r.Content.ReadAsStringAsync();
}

// POST que retorna objeto completo (solo para Login)
public async Task<TRes?> PostObjectAsync<TReq, TRes>(string url, TReq body)
{
    var r = await _http.PostAsync(url, Json(body));
    r.EnsureSuccessStatusCode();
    return JsonConvert.DeserializeObject<TRes>(await r.Content.ReadAsStringAsync());
}

public async Task<bool> PutAsync<T>(string url, T body)
{
    var r = await _http.PutAsync(url, Json(body));
    return r.IsSuccessStatusCode;
}

private static StringContent Json<T>(T obj) =>
    new(JsonConvert.SerializeObject(obj), Encoding.UTF8, "application/json");
}
```

Paso 6 — AuthService.cs

```
using SidApiClient.Core.Helpers;
using SidApiClient.Core.Models.DTOs;

namespace SidApiClient.Core.Services;

public class AuthService(ApiClient client)
{
    public async Task LoginAsync(string username, string password)
    {
        var result = await client.PostObjectAsync<LoginRequest, dynamic>(
            "/F0_Acceso/Login",
            new LoginRequest { Username = username, Password = password });
        string token = result?.token
            ?? throw new InvalidOperationException("Login fallido: token no recibido.");
        client.SetToken(token);
    }

    // POST retorna string con el ID de la configuración
    public Task<string?> ConfigurarEmpresaAsync(ConfiguracionEmpresaDTO config)
        => client.PostAsync("/F0_Acceso/Configuracion", config);

    public Task<ConfiguracionEmpresaDTO?> ObtenerConfiguracionAsync()
        => client.GetAsync<ConfiguracionEmpresaDTO>("/F0_Acceso/Configuracion");

    // Debe invocarse cada 60 s en producción
    public Task<string?> EnviarEstadoSidAsync(string estado)
        => client.PostAsync("/F1_ConfiguracionInicial/EstadoSID", new { estado });
}
```

Paso 7 — Program.cs

```
using SidApiClient.Core.Helpers;
using SidApiClient.Core.Models.DTOs;
using SidApiClient.Core.Services;
```

```
var client = new ApiClient("https://lapem.cfe.gob.mx/sid_capacitacion");
var authSvc = new AuthService(client);

await authSvc.LoginAsync("tu_usuario", "tu_contraseña");
Console.WriteLine("✓ Login exitoso.");

var configId = await authSvc.ConfigurarEmpresaAsync(new ConfiguracionEmpresaDTO
{
    NumeroIntentosPermitidos = 3,
    Muestreo = new MuestreoDTO { TipoMuestreo = "NORMAL", NumeroFijoMuestras = 5 }
});
Console.WriteLine($"✓ Configuración creada. ID: {configId}");

var config = await authSvc.ObtenerConfiguracionAsync();
Console.WriteLine($"✓ Intentos permitidos: {config?.NumeroIntentosPermitidos}");

await authSvc.EnviarEstadoSidAsync("EN_ESPERA");
Console.WriteLine("✓ Estado SID enviado.");
```

Paso 8 — Ejecutar y hacer commit

19. Guardar todos los archivos.
20. Ejecutar: `dotnet run --project SidApiClient.App`
21. Verificar en consola: Login exitoso + ID de configuración impreso.
22. Hacer commit:

```
git add .
git commit -m "Pl: AuthService y configuración inicial"
git push
```

Criterios de evaluación:

- Proyecto compila y ejecuta sin errores.
- Login obtiene token JWT y lo almacena automáticamente.
- ConfigurarEmpresa retorna string con el ID (no bool ni objeto).
- Commit en repositorio GitHub privado compartido con los revisores.

Práctica 2: Gestión de Instrumentos de Medición

GET /F1_ConfiguracionInicial/Instrumento

GET /F1_ConfiguracionInicial/Instrumento/{id}

POST /F1_ConfiguracionInicial/Instrumento

PUT /F1_ConfiguracionInicial/Instrumento

DTO(s)

```
public class InstrumentoDTO
{
    public string? Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string NumeroSerie { get; set; } = string.Empty;
    public string FechaCalibracion { get; set; } = string.Empty;
```

```
public string FechaVencimientoCalibracion { get; set; } = string.Empty;  
public string? UrlArchivo { get; set; }  
public string Estatus { get; set; } = "ACTIVO";  
}
```

Servicio

```
public class InstrumentoService(ApiClient client)  
{  
    public Task<List<InstrumentoDTO>?> ObtenerAsync()  
        => client.GetAsync<List<InstrumentoDTO>>("/Fl_ConfiguracionInicial/Instrumento");  
  
    public Task<InstrumentoDTO?> ObtenerPorIdAsync(string id)  
        => client.GetAsync<InstrumentoDTO>($" /Fl_ConfiguracionInicial/Instrumento/{id}");  
  
    // POST retorna string con el ID creado  
    public Task<string?> RegistrarAsync(InstrumentoDTO dto)  
        => client.PostAsync("/Fl_ConfiguracionInicial/Instrumento", dto);  
  
    public Task<bool> ActualizarAsync(InstrumentoDTO dto)  
        => client.PutAsync("/Fl_ConfiguracionInicial/Instrumento", dto);  
}
```

Notas:

- Validar que FechaVencimientoCalibracion sea futura.
- Guardar el ID retornado para la Práctica 11 (AgregaResultadoPrueba).

Commit requerido:

```
git add .  
git commit -m "P2: InstrumentoService implementado"  
git push
```

Práctica 3: Administración de Prototipos

GET /Fl_ConfiguracionInicial/Prototipo

POST /Fl_ConfiguracionInicial/Prototipo

PUT /Fl_ConfiguracionInicial/Prototipo

DTO(s)

```
public class PrototipoDTO  
{  
    public string? Id { get; set; }  
    public string Numero { get; set; } = string.Empty;  
    public string FechaEmision { get; set; } = string.Empty;  
    public string FechaVencimiento { get; set; } = string.Empty;  
    public string? UrlArchivo { get; set; }  
    public string Estatus { get; set; } = "ACTIVO";  
}
```

Servicio

```
public class PrototipoService(ApiClient client)  
{
```

```
public Task<List<PrototipoDTO>>> ObtenerAsync()  
=> client.GetAsync<List<PrototipoDTO>>>("/Fl_ConfiguracionInicial/Prototipo");  
  
public Task<string?> RegistrarAsync(PrototipoDTO dto)  
=> client.PostAsync("/Fl_ConfiguracionInicial/Prototipo", dto);  
  
public Task<bool> ActualizarAsync(PrototipoDTO dto)  
=> client.PutAsync("/Fl_ConfiguracionInicial/Prototipo", dto);  
}
```

Notas:

- Guardar el ID del prototipo para usarlo en ProductoService (P5).

Commit requerido:

```
git add .  
git commit -m "P3: PrototipoService implementado"  
git push
```

Práctica 4: Gestión de Normas

GET

/Fl_ConfiguracionInicial/Norma

POST

/Fl_ConfiguracionInicial/Norma

PUT

/Fl_ConfiguracionInicial/Norma

DTO(s)

```
public class NormaDTO  
{  
    public string? Id { get; set; }  
    public string Clave { get; set; } = string.Empty;  
    public string Nombre { get; set; } = string.Empty;  
    public string Edicion { get; set; } = string.Empty;  
    public string Estatus { get; set; } = "ACTIVO";  
    public bool EsCFE { get; set; }  
}
```

Servicio

```
public class NormaService(ApiClient client)  
{  
    public Task<List<NormaDTO>>> ObtenerAsync()  
=> client.GetAsync<List<NormaDTO>>>("/Fl_ConfiguracionInicial/Norma");  
  
    public async Task<List<NormaDTO>>> ObtenerCFEAsync()  
    {  
        var all = await ObtenerAsync();  
        return all?.Where(n => n.EsCFE).ToList() ?? new();  
    }  
  
    public Task<string?> RegistrarAsync(NormaDTO dto)  
=> client.PostAsync("/Fl_ConfiguracionInicial/Norma", dto);  
  
    public Task<bool> ActualizarAsync(NormaDTO dto)  
=> client.PutAsync("/Fl_ConfiguracionInicial/Norma", dto);  
}
```

Notas:

- Guardar el ID de la norma para asociarla al producto en P5.

Commit requerido:

```
git add .
git commit -m "P6: NormaService implementado"
git push
```

Práctica 5: Configuración de Pruebas

GET /Fl_ConfiguracionInicial/Prueba
POST /Fl_ConfiguracionInicial/Prueba
PUT /Fl_ConfiguracionInicial/Prueba
POST /Fl_ConfiguracionInicial/OtrasPruebasYDocumentos

DTO(s)

```
public class PruebaDTO
{
    public string? Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Estatus { get; set; } = "ACTIVO";
    // RUTINA | TIPO | ESPECIAL
    public string TipoPrueba { get; set; } = "RUTINA";
    // NUMERICO | APROBADO RECHAZADO
    public string TipoResultado { get; set; } = "NUMERICO";
}

public class OtrasPruebasYDocumentosDTO
{
    public string? Id { get; set; }
    // Politica|ManualUsuario|InstructivoTecnico|Contrato|Otro
    public string TipoDocumento { get; set; } = string.Empty;
    public string DescripcionDocumento { get; set; } = string.Empty;
    public string UrlArchivo { get; set; } = string.Empty;
    public string Estatus { get; set; } = "ACTIVO";
    public string Vigencia { get; set; } = string.Empty;
}
```

Servicio

```
public class PruebaService(ApiClient client)
{
    public Task<List<PruebaDTO>?> ObtenerAsync()
        => client.GetAsync<List<PruebaDTO>>("/Fl_ConfiguracionInicial/Prueba");

    public Task<string?> RegistrarAsync(PruebaDTO dto)
        => client.PostAsync("/Fl_ConfiguracionInicial/Prueba", dto);

    public Task<bool> ActualizarAsync(PruebaDTO dto)
        => client.PutAsync("/Fl_ConfiguracionInicial/Prueba", dto);

    public Task<string?> RegistrarDocumentoAsync(OtrasPruebasYDocumentosDTO dto)
```

```
=> client.PostAsync("/Fl_ConfiguracionInicial/OtrasPruebasYDocumentos", dto);  
}
```

Notas:

- Guardar el ID de cada prueba para asociarla al producto en P5.

Commit requerido:

```
git add .  
git commit -m "P7: PruebaService implementado"  
git push
```

Práctica 6: Administración de Productos

GET /Fl_ConfiguracionInicial/Producto
POST /Fl_ConfiguracionInicial/Producto
PUT /Fl_ConfiguracionInicial/Producto

DTO(s)

```
public class AgregarProductoDTO  
{  
    public string? Id { get; set; }  
    public string CodigoFabricante { get; set; } = string.Empty;  
    public string Descripcion { get; set; } = string.Empty;  
    public string DescripcionCorta { get; set; } = string.Empty;  
    public string TipoFabricacion { get; set; } = "SERIE";  
    public string Unidad { get; set; } = string.Empty;  
    public string Norma { get; set; } = string.Empty;  
    public string Prototipo { get; set; } = string.Empty;  
    public string Estatus { get; set; } = "ACTIVO";  
    public List<string> Pruebas { get; set; } = new();  
}
```

Servicio

```
public class ProductoService(ApiClient client)  
{  
    public Task<List<AgregarProductoDTO>?> ObtenerAsync()  
        => client.GetAsync<List<AgregarProductoDTO>>("/Fl_ConfiguracionInicial/Producto");  
  
    public Task<string?> RegistrarAsync(AgregarProductoDTO dto)  
        => client.PostAsync("/Fl_ConfiguracionInicial/Producto", dto);  
  
    public Task<bool> ActualizarAsync(AgregarProductoDTO dto)  
        => client.PutAsync("/Fl_ConfiguracionInicial/Producto", dto);  
}
```

Notas:

- Pruebas es lista de IDs de PruebaDTO registrados en P7.
- Guardar el ID del producto para P9 (OrdenFabricacion) y P4 (ValorReferencia).

Commit requerido:

```
git add .
git commit -m "P5: ProductoService implementado"
git push
```

Práctica 7: Gestión de Valores de Referencia

GET /F1_ConfiguracionInicial/ValorReferencia
POST /F1_ConfiguracionInicial/ValorReferencia
PUT /F1_ConfiguracionInicial/ValorReferencia

DTO(s)

```
public class ValorReferenciaDTO
{
    public string? Id { get; set; }
    public string IdProducto { get; set; } = string.Empty;
    public string IdPrueba { get; set; } = string.Empty;
    public double Valor { get; set; }
    public double? Valor2 { get; set; }
    public string Unidad { get; set; } = string.Empty;
    // IGUAL | MAYOR | MENOR | ENTRE
    public string Comparacion { get; set; } = "ENTRE";
}
```

Servicio

```
public class ValorReferenciaService(ApiClient client)
{
    public Task<List<ValorReferenciaDTO>>? ObtenerAsync()
    =>
    client.GetAsync<List<ValorReferenciaDTO>>("/F1_ConfiguracionInicial/ValorReferencia");

    public Task<string?> RegistrarAsync(ValorReferenciaDTO dto)
    => client.PostAsync("/F1_ConfiguracionInicial/ValorReferencia", dto);

    public Task<bool> ActualizarAsync(ValorReferenciaDTO dto)
    => client.PutAsync("/F1_ConfiguracionInicial/ValorReferencia", dto);
}
```

Notas:

- Para Comparacion=ENTRE: Valor=mínimo, Valor2=máximo.

Commit requerido:

```
git add .
git commit -m "P4: ValorReferenciaService implementado"
git push
```

Práctica 8: Gestión de Contratos CFE

GET /F2_PreparacionFabricacion/Contratos
POST /F2_PreparacionFabricacion/Contratos

PUT /F2_PreparacionFabricacion/Contratos

DTO(s)

```
public class ContratoDTO
{
    public string? Id { get; set; }
    // SUMINISTRO | OBRA
    public string TipoContrato { get; set; } = "SUMINISTRO";
    public string NoContrato { get; set; } = string.Empty;
    public string Estatus { get; set; } = "ACTIVO";
}
```

Servicio

```
public class ContratoService(ApiClient client)
{
    public Task<List<ContratoDTO>?> ObtenerAsync()
        => client.GetAsync<List<ContratoDTO>>("/F2_PreparacionFabricacion/Contratos");

    public Task<string?> RegistrarAsync(ContratoDTO dto)
        => client.PostAsync("/F2_PreparacionFabricacion/Contratos", dto);

    public Task<bool> ActualizarAsync(ContratoDTO dto)
        => client.PutAsync("/F2_PreparacionFabricacion/Contratos", dto);
}
```

Notas:

- Guardar el ID del contrato y número de partida para el detalle en P9.

Commit requerido:

```
git add .
git commit -m "P8: ContratoService implementado"
git push
```

Práctica 9: Órdenes de Fabricación

POST /F2_PreparacionFabricacion/OrdenFabricacion**PUT** /F2_PreparacionFabricacion/OrdenFabricacion**GET** /F2_PreparacionFabricacion/OrdenFabricacion/{IdOrdenFabricacion}

DTO(s)

```
public class DetalleFabricacionDTO
{
    public string ContratoId { get; set; } = string.Empty;
    public string TipoContrato { get; set; } = string.Empty;
    public string PartidaContratoId { get; set; } = string.Empty;
    public string DescripcionPartida { get; set; } = string.Empty;
    public string Unidad { get; set; } = string.Empty;
    public double CantidadOriginalContrato { get; set; }
    public double CantidadAFabricar { get; set; }
}
```

```
public class OrdenFabricacionDTO
{
    public string? Id { get; set; }
    public string ClaveOrdenFabricacion { get; set; } = string.Empty;
    public string LoteFabricacion { get; set; } = string.Empty;
    public string IdProducto { get; set; } = string.Empty;
    public List<DetalleFabricacionDTO> DetalleFabricacion { get; set; } = new();
}
```

Servicio

```
public class OrdenFabricacionService(ApiClient client)
{
    public Task<string?> CrearAsync(OrdenFabricacionDTO dto)
        => client.PostAsync("/F2_PreparacionFabricacion/OrdenFabricacion", dto);

    public Task<bool> ActualizarAsync(OrdenFabricacionDTO dto)
        => client.PutAsync("/F2_PreparacionFabricacion/OrdenFabricacion", dto);

    public Task<OrdenFabricacionDTO?> ObtenerAsync(string id)
        => client.GetAsync<OrdenFabricacionDTO>($" /F2_PreparacionFabricacion/OrdenFabricacion/{id}");
}
```

Notas:

- Guardar el ID de la orden para crear el expediente en P10.

Commit requerido:

```
git add .
git commit -m "P9: OrdenFabricacionService implementado"
git push
```

Práctica 10: Expedientes de Pruebas

POST	/F2_PreparacionFabricacion/ExpedientePruebas
PUT	/F2_PreparacionFabricacion/ExpedientePruebas
GET	/F2_PreparacionFabricacion/ExpedientePruebas
PUT	/F2_PreparacionFabricacion/AgregaMuestraExpediente/{Expediente}
PUT	/F2_PreparacionFabricacion/QuitarMuestraExpediente/{Expediente}/{muestra}

DTO(s)

```
public class NuevoExpedienteInspeccionDTO
{
    public string? Id { get; set; }
    public string ClaveExpediente { get; set; } = string.Empty;
    public string OrdenFabricacion { get; set; } = string.Empty;
    public int CantidadMuestras { get; set; }
    public int MaximoRechazos { get; set; }
    public List<string> Muestras { get; set; } = new();
}
```

Servicio

```
public class ExpedienteService(ApiClient client)
{
    public Task<string?> CrearAsync(NuevoExpedienteInspeccionDTO dto)
        => client.PostAsync("/F2_PreparacionFabricacion/ExpedientePruebas", dto);

    public Task<bool> ActualizarAsync(NuevoExpedienteInspeccionDTO dto)
        => client.PutAsync("/F2_PreparacionFabricacion/ExpedientePruebas", dto);

    public Task<bool> AgregarMuestraAsync(string clave, string muestra)
        => client.PutAsync($"/F2_PreparacionFabricacion/AgregaMuestraExpediente/{clave}",
        new { muestra });

    public Task<bool> QuitarMuestraAsync(string clave, string muestra)
        =>
        client.PutAsync($"/F2_PreparacionFabricacion/QuitarMuestraExpediente/{clave}/{muestra}", new
        { });
}
```

Notas:

- La ClaveExpediente (no el Id) es el identificador en todos los endpoints de F3 y F4.

Commit requerido:

```
git add .
git commit -m "P10: ExpedienteService implementado"
git push
```

Práctica 11: Ejecución de Pruebas y Liberación

PUT	/F3_Pruebas/AgregaResultadoPrueba
GET	/F3_Pruebas/PruebasNoSatisfactorias/{Expediente}
GET	/F3_Pruebas/ValidacionExpediente/{Expediente}
PUT	/F3_Pruebas/TerminarPruebasExpediente/{Expediente}
PUT	/F4_Liberacion/CrearAvisoPrueba/{Expediente}
GET	/F4_Liberacion/ConsultaAvisos/{Expediente}

DTO(s)

```
public class AgregaResultadoPruebaDTO
{
    public string IdPrueba { get; set; } = string.Empty;
    public string IdValorReferencia { get; set; } = string.Empty;
    public string FechaPrueba { get; set; } = string.Empty;
    public string OperadorPrueba { get; set; } = string.Empty;
    public string IdInstrumentoMedicion { get; set; } = string.Empty;
    public double ValorMedido { get; set; }
    // APROBADO | RECHAZADO
    public string Resultado { get; set; } = "APROBADO";
    public int NumeroIntento { get; set; } = 1;
}
```

Servicio

```
public class PruebasEjecucionService(ApiClient client)
{
```



```
public Task<bool> AgregarResultadoAsync(AgregaResultadoPruebaDTO dto)
    => client.PutAsync("/F3_Pruebas/AgregaResultadoPrueba", dto);

public Task<object?> PruebasNoSatisfactoriasAsync(string clave)
    => client.GetAsync<object>($" /F3_Pruebas/PruebasNoSatisfactorias/{clave}");

public Task<object?> ValidarExpedienteAsync(string clave)
    => client.GetAsync<object>($" /F3_Pruebas/ValidacionExpediente/{clave}");

public Task<bool> TerminarPruebasAsync(string clave)
    => client.PutAsync($" /F3_Pruebas/TerminarPruebasExpediente/{clave}", new { });

public Task<bool> CrearAvisoPruebaAsync(string clave)
    => client.PutAsync($" /F4_Liberacion/CrearAvisoPrueba/{clave}", new { });

public Task<object?> ConsultarAvisosAsync(string clave)
    => client.GetAsync<object>($" /F4_Liberacion/ConsultaAvisos/{clave}");
}
```

Notas:

- Flujo F3: AgregarResultado (×muestras×pruebas) → TerminarPruebas → F4.
- Usar PruebasNoSatisfactorias para reintentos (hasta NumeroIntentosPermitidos).

Commit requerido:

```
git add .
git commit -m "Pl1: PruebasEjecucionService - flujo completo"
git push
```



4. Estructura Final del Proyecto

```
SidApiClient/  
├── SidApiClient.Core/  
│   ├── Models/DTOs/  
│   │   ├── LoginRequest.cs  
│   │   ├── ConfiguracionEmpresaDTO.cs  
│   │   ├── InstrumentoDTO.cs  
│   │   ├── PrototipoDTO.cs  
│   │   ├── ValorReferenciaDTO.cs  
│   │   ├── AgregarProductoDTO.cs  
│   │   ├── NormaDTO.cs  
│   │   ├── PruebaDTO.cs  
│   │   ├── ContratoDTO.cs  
│   │   ├── OrdenFabricacionDTO.cs  
│   │   ├── DetalleFabricacionDTO.cs  
│   │   ├── NuevoExpedienteInspeccionDTO.cs  
│   │   └── AgregaResultadoPruebaDTO.cs  
│   └── Services/  
│       ├── AuthService.cs  
│       ├── InstrumentoService.cs  
│       ├── PrototipoService.cs  
│       ├── ValorReferenciaService.cs  
│       ├── ProductoService.cs  
│       ├── NormaService.cs  
│       ├── PruebaService.cs  
│       ├── ContratoService.cs  
│       ├── OrdenFabricacionService.cs  
│       ├── ExpedienteService.cs  
│       └── PruebasEjecucionService.cs  
├── Helpers/  
│   └── ApiClient.cs  
├── SidApiClient.App/  
│   ├── Program.cs  
│   └── appsettings.json  
└── SidApiClient.sln
```